



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 9: Semáforo binario y contador

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 2 de octubre de 2026

Índice general

Índice de figuras	III
Índice de cuadros	IV
1. Semáforo binario: cuenta bancaria	1
1.1. Entradas y procesamiento	1
1.2. Prueba de escritorio	1
1.3. Evidencia y resultados	2
2. Semáforo contador: impresoras	4
2.1. Entradas y procesamiento	4
2.2. Prueba de escritorio	4
2.3. Evidencia y resultados	5
3. Comparación y conclusión	7
3.1. Comparación	7
3.2. Conclusión	7

Índice de figuras

1.1. Código y resultados del semáforo binario con Semaphore(1).	3
2.1. Código y resultados del semáforo contador con Semaphore(3).	6

Índice de cuadros

1.1. Prueba de escritorio del semáforo binario.	2
2.1. Prueba de escritorio del semáforo contador.	5
3.1. Comparación entre los dos semáforos utilizados.	7

1. Semáforo binario: cuenta bancaria

Instrucción. Crear una cuenta bancaria compartida con saldo inicial de \$1000 y cinco hilos que realicen depósitos y retiros. El acceso al saldo debe estar protegido por un semáforo binario con un solo permiso.

1.1 Entradas y procesamiento

El programa utiliza un saldo compartido de \$1000 y un objeto de la clase Semaphore con un permiso:

```
private static final Semaphore sem = new Semaphore(1);
```

El método `acquire()` solicita el permiso. Si otro hilo ya lo tiene, el hilo que intenta entrar queda bloqueado hasta que el permiso sea liberado. Al terminar la operación sobre el saldo se utiliza `release()` para devolver el permiso.

Los cinco hilos realizan tres depósitos de \$200, \$300 y \$50, y dos retiros de \$150 y \$100. Por lo tanto, el saldo final esperado es

$$1000 + 200 - 150 + 300 - 100 + 50 = 1300.$$

En esta implementación, `acquire()` cumple el papel de la operación `P()` estudiada en clase y `release()` cumple el papel de `V()`.

1.2 Prueba de escritorio

Para observar la exclusión mutua se consideran dos hilos: A deposita \$200 y B retira \$150.

Cuadro 1.1: Prueba de escritorio del semáforo binario.

Paso	Acción	Permisos	Resultado
0	Estado inicial	1	El semáforo está disponible.
1	A ejecuta acquire()	0	A obtiene el único permiso.
2	B ejecuta acquire()	0	B queda esperando.
3	A deposita \$200	0	El saldo cambia de \$1000 a \$1200.
4	A ejecuta release()	1	El permiso queda disponible para B.
5	B obtiene el permiso	0	B entra y retira \$150.
6	B ejecuta release()	1	El semáforo vuelve a quedar disponible.

1.3 Evidencia y resultados

La Figura 1.1 muestra en una sola captura el código utilizado y varias ejecuciones del programa. El orden de los hilos cambia entre ejecuciones, pero en todos los casos el saldo final mostrado es \$1300. Esto confirma que cada hilo obtiene el único permiso antes de modificar la variable compartida.

2. Semáforo contador: impresoras

Instrucción. Simular tres impresoras compartidas por ocho hilos. El semáforo debe permitir como máximo tres impresiones simultáneas.

2.1 Entradas y procesamiento

El programa representa las tres impresoras mediante un semáforo con tres permisos:

```
private static final Semaphore impresoras = new Semaphore(3);
```

Cada hilo ejecuta `impresoras.acquire()` antes de imprimir. Los tres primeros hilos que obtienen un permiso pueden continuar; si un cuarto hilo intenta entrar cuando los tres permisos están ocupados, queda bloqueado.

Después de simular la impresión mediante `Thread.sleep(1000)`, el hilo ejecuta `impresoras.release()` y devuelve su permiso. De esta forma otro hilo que estaba esperando puede comenzar.

2.2 Prueba de escritorio

Para la prueba se consideran dos permisos y cuatro hilos A, B, C y D.

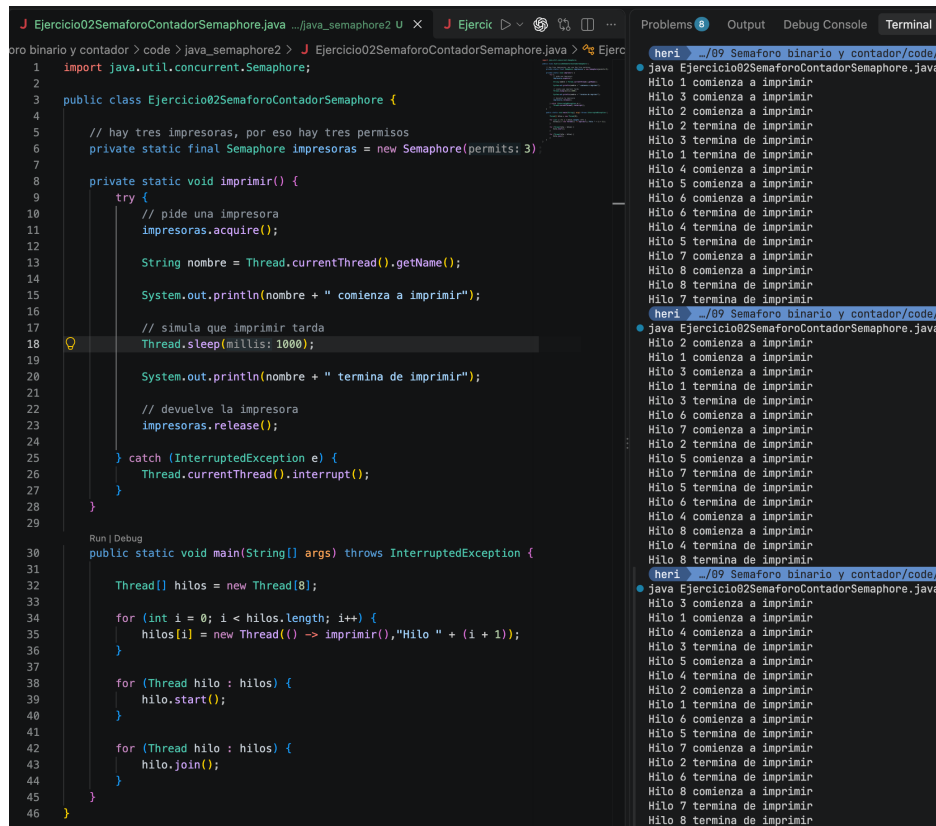
Cuadro 2.1: Prueba de escritorio del semáforo contador.

Paso	Acción	Permisos	Resultado
0	Estado inicial	2	Hay dos impresoras disponibles.
1	A ejecuta <code>acquire()</code>	1	A comienza a imprimir.
2	B ejecuta <code>acquire()</code>	0	B comienza a imprimir.
3	C ejecuta <code>acquire()</code>	0	C queda esperando.
4	D ejecuta <code>acquire()</code>	0	D queda esperando.
5	A ejecuta <code>release()</code>	1	Uno de los hilos bloqueados puede continuar.
6	C obtiene el permiso	0	C comienza a imprimir.
7	B ejecuta <code>release()</code>	1	D puede obtener el siguiente permiso.
8	C y D terminan	2	Los dos permisos vuelven a estar disponibles.

2.3 Evidencia y resultados

La Figura 2.1 muestra el código y varias ejecuciones en la misma captura. En la terminal se observa que varios hilos comienzan a imprimir antes de que terminen los anteriores, pero el semáforo solo dispone de tres permisos. Por ello, como máximo tres hilos pueden encontrarse usando una impresora al mismo tiempo. El orden cambia entre ejecuciones porque depende de la planificación de los hilos.

Parte 2. Semáforo contador: impresoras



```
1 import java.util.concurrent.Semaphore;
2
3 public class Ejercicio02SemaforoContadorSemaphore {
4
5     // hay tres impresoras, por eso hay tres permisos
6     private static final Semaphore impresoras = new Semaphore(permits: 3);
7
8     private static void imprimir() {
9         try {
10             // pide una impresora
11             impresoras.acquire();
12
13             String nombre = Thread.currentThread().getName();
14
15             System.out.println(nombre + " comienza a imprimir");
16
17             // simula que imprimir tarda
18             Thread.sleep(millis: 1000);
19
20             System.out.println(nombre + " termina de imprimir");
21
22             // devuelve la impresora
23             impresoras.release();
24         } catch (InterruptedException e) {
25             Thread.currentThread().interrupt();
26         }
27     }
28
29
30     public static void main(String[] args) throws InterruptedException {
31
32         Thread[] hilos = new Thread[8];
33
34         for (int i = 0; i < hilos.length; i++) {
35             hilos[i] = new Thread(() -> imprimir(), "Hilo " + (i + 1));
36         }
37
38         for (Thread hilo : hilos) {
39             hilo.start();
40         }
41
42         for (Thread hilo : hilos) {
43             hilo.join();
44         }
45     }
46 }
```

heri /09 Semaforo binario y contador/code/

- java Ejercicio02SemaforoContadorSemaphore.java
- Hilo 1 comienza a imprimir
- Hilo 3 comienza a imprimir
- Hilo 2 comienza a imprimir
- Hilo 2 termina de imprimir
- Hilo 3 termina de imprimir
- Hilo 1 termina de imprimir
- Hilo 4 comienza a imprimir
- Hilo 5 comienza a imprimir
- Hilo 6 comienza a imprimir
- Hilo 6 termina de imprimir
- Hilo 4 termina de imprimir
- Hilo 5 termina de imprimir
- Hilo 7 comienza a imprimir
- Hilo 8 comienza a imprimir
- Hilo 8 termina de imprimir
- Hilo 7 termina de imprimir

heri /09 Semaforo binario y contador/code/

- java Ejercicio02SemaforoContadorSemaphore.java
- Hilo 2 comienza a imprimir
- Hilo 1 comienza a imprimir
- Hilo 3 comienza a imprimir
- Hilo 1 termina de imprimir
- Hilo 3 termina de imprimir
- Hilo 2 termina de imprimir
- Hilo 6 comienza a imprimir
- Hilo 7 comienza a imprimir
- Hilo 2 termina de imprimir
- Hilo 5 comienza a imprimir
- Hilo 4 comienza a imprimir
- Hilo 8 comienza a imprimir
- Hilo 4 termina de imprimir
- Hilo 8 termina de imprimir

heri /09 Semaforo binario y contador/code/

- java Ejercicio02SemaforoContadorSemaphore.java
- Hilo 3 comienza a imprimir
- Hilo 1 comienza a imprimir
- Hilo 4 comienza a imprimir
- Hilo 3 termina de imprimir
- Hilo 5 comienza a imprimir
- Hilo 4 termina de imprimir
- Hilo 2 comienza a imprimir
- Hilo 1 termina de imprimir
- Hilo 6 comienza a imprimir
- Hilo 5 termina de imprimir
- Hilo 7 comienza a imprimir
- Hilo 2 termina de imprimir
- Hilo 6 termina de imprimir
- Hilo 8 comienza a imprimir
- Hilo 7 termina de imprimir
- Hilo 8 termina de imprimir

Figura 2.1: Código y resultados del semáforo contador con Semaphore(3).

3. Comparación y conclusión

3.1 Comparación

Cuadro 3.1: Comparación entre los dos semáforos utilizados.

Aspecto	Binario	Contador
Creación	<code>new Semaphore(1)</code>	<code>new Semaphore(3)</code>
Permisos iniciales	1	3
Recurso controlado	Saldo bancario	Tres impresoras
Máximo de hilos simultáneos	1	3
Solicitar permiso	<code>acquire()</code>	<code>acquire()</code>
Liberar permiso	<code>release()</code>	<code>release()</code>
Bloqueo	Cuando el permiso está ocupado	Cuando los tres permisos están ocupados

3.2 Conclusión

La clase Semaphore de Java permite aplicar directamente el mecanismo estudiado en clase. Con un permiso se obtiene exclusión mutua sobre la cuenta bancaria; con tres permisos se permite que hasta tres hilos utilicen recursos equivalentes de forma concurrente. En ambos ejercicios, `acquire()` solicita un permiso y puede bloquear al hilo, mientras que `release()` devuelve el permiso para que otro hilo pueda continuar.

Los resultados observados coinciden con el comportamiento esperado. En la cuenta bancaria el orden de ejecución puede cambiar, pero el saldo final se mantiene en \$1300. En el ejercicio de impresoras el orden también varía, pero

la cantidad de hilos que puede avanzar simultáneamente queda limitada por los tres permisos del semáforo.